

Actualizar arrays en el estado

Los *arrays* son mutables en JavaScript, pero deberían tratarse como inmutables cuando los almacenas en el estado. Al igual que los objetos, cuando quieras actualizar un *array* almacenado en el estado, necesitas crear uno nuevo (o hacer una copia de uno existente) y luego asignar el estado para que utilice este nuevo *array*.

Aprenderás

- Cómo añadir, eliminar o cambiar elementos en un *array* en el estado de React
- Cómo actualizar un objeto dentro de un *array*
- Cómo copiar un *array* de forma menos repetitiva con Immer

Actualizar *arrays* sin mutación

En JavaScript, los *arrays* son solo otro tipo de objeto. [Como con los objetos](#), **deberías tratar los *arrays* en el estado de React como si fueran de solo lectura**. Esto significa que no deberías reasignar elementos dentro de un *array* como `arr[0] = 'pájaro'`, y tampoco deberías usar métodos que puedan mutar el *array*, como `push()` y `pop()`.

En su lugar, cada vez que quieras actualizar un *array*, querrás pasar un *nuevo array* a la función de asignación de estado. Para hacerlo, puedes crear un nuevo *array* a partir del *array* original en el estado si llamas a sus métodos que no lo muten como `filter()` y `map()`. Luego puedes asignar el estado a partir del nuevo *array* resultante.

Aquí hay una tabla de referencia con las operaciones más comunes con *arrays*. Cuando se trata de *arrays* dentro del estado de React, necesitarás evitar los métodos de la columna izquierda, y en su lugar es preferible usar los métodos de la columna derecha.

	evita (muta el <i>array</i>)	preferido (devuelve un nuevo <i>array</i>)
añadir	push, unshift	concat, [...arr] operador de propagación (ejemplo)
eliminar	pop, shift, splice	filter, slice (ejemplo)
reemplazar	splice, arr[i] = ... asigna	map (ejemplo)
ordenar	reverse, sort	copia el <i>array</i> primero (ejemplo)

Como alternativa, puedes [usar Immer](#) el cual te permite usar métodos de ambas columnas.

Atención

Desafortunadamente, [slice](#) y [splice](#) tienen nombres muy similares pero son muy diferentes:

- [slice](#) te permite copiar un *array* o una parte del mismo.
- [splice](#) **muta** el *array* (para insertar o eliminar elementos).

En React, estarás usando [slice](#) (no [splice](#)!) mucho más seguido porque no quieres mutar objetos o *arrays* en el estado. [Actualizar objetos](#) explica qué es mutación y por qué no se recomienda para el estado.

Añadir a un *array*

`push()` muta un *array*, lo cual no queremos:

App.js

Descargar Reiniciar

```
13     value={name}
14     onChange={e => setName(e.target.value)}
15   />
16   <button onClick={() => {
17     artists.push({
18       id: nextId++,
19       name: name,
20     });
21   }}>Añadir</button>
22   <ul>
23     {artists.map(artist => (
24       <li key={artist.id}>{artist.name}</li>
```

▼ Mostrar más

Escultores inspiradores:

En su lugar, crea un *nuevo array* que contenga los elementos existentes y un nuevo elemento al final. Hay múltiples formas de hacerlo, pero la más fácil es usar la

sintaxis ... de propagación en *arrays*:

```
setArtists( // Reemplaza el estado
  [ // con el nuevo _array_
    ...artists, // el cual contiene todos los elementos antiguos
    { id: nextId++, name: name } // y un nuevo elemento al final
  ]
);
```

Ahora funciona correctamente:

App.js

⬇ Descargar ⌛ Reiniciar 🔗

```
19         { id: nextId++, name: name }
20     });
21   }}>Añadir</button>
22   <ul>
23     {artists.map(artist => (
24       <li key={artist.id}>{artist.name}</li>
25     ))}
26   </ul>
27 </>
28 );
29 }
30
```

▼ Mostrar más

El operador de propagación también te permite anteponer un elemento al colocarlo *antes* del original `...artists`:

```
setArtists([
  { id: nextId++, name: name },
  ...artists // Coloca los elementos antiguos al final
]);
```

De esta forma, el operador de propagación puede hacer el trabajo tanto de `push()` añadiendo al final del *array* como de `unshift()` agregando al comienzo del *array*. ¡Pruébalo en el editor de arriba!

Eliminar elementos de un *array*

La forma más fácil de eliminar un elemento de un *array* es *filtrarlo*. En otras palabras, producirás un nuevo *array* que no contendrá ese elemento. Para hacerlo, usa el método `filter`, por ejemplo:

App.js

⬇ Descargar ↺ Reiniciar 🔗

```
1  import { useState } from 'react';
2
3  let initialArtists = [
4    { id: 0, name: 'Marta Colvin Andrade' },
5    { id: 1, name: 'Lamidi Olonade Fakeye'},
6    { id: 2, name: 'Louise Nevelson'},
7  ];
8
9  export default function List() {
10    const [artists, setArtists] = useState(
11      initialArtists
12    );
```

▼ Mostrar más

Haz click en el botón “Eliminar” varias veces, y mira su controlador de clics.

```
setArtists(  
  artists.filter(a => a.id !== artist.id)  
);
```

Aquí, `artists.filter(a => a.id !== artist.id)` significa “crea un nuevo *array* conformado por aquellos `artists` cuyos IDs son diferentes de `artist.id`”. En otras palabras, el botón “Eliminar” de cada artista filtrará a ese artista del *array* y luego solicitará un rerenderizado con el *array* resultante. Ten en cuenta que `filter` no modifica el *array* original.

Transformar un *array*

Si deseas cambiar algunos o todos los elementos del *array*, puedes usar `map()` para crear un **nuevo** *array*. La función que pasarás a `map` puede decidir qué hacer con cada elemento, en función de sus datos o su índice (o ambos).

En este ejemplo, un *array* contiene las coordenadas de dos círculos y un cuadrado. Cuando presionas el botón, mueve solo los círculos 50 píxeles hacia abajo. Lo hace produciendo un nuevo *array* de datos usando `map()` :

App.js

[⬇ Descargar](#) [🔄 Reiniciar](#) [🔗](#)

```
import { useState } from 'react';

let initialShapes = [
  { id: 0, type: 'circle', x: 50, y: 100 },
  { id: 1, type: 'square', x: 150, y: 100 },
  { id: 2, type: 'circle', x: 250, y: 100 },
];

export default function ShapeEditor() {
  const [shapes, setShapes] = useState(
    initialShapes
  );
```

▼ Mostrar más

Reemplazar elementos en un *array*

Es particularmente común querer reemplazar uno o más elementos en un *array*. Las asignaciones como `arr[0] = 'pájaro'` están mutando el *array* original, por lo que para esto también querrás usar `map`.

Para reemplazar un elemento, crea un nuevo *array* con `map`. Dentro de la llamada a `map`, recibirás el índice del elemento como segundo argumento. Úsalo para decidir si devolver el elemento original (el primer argumento) o algo más:

[App.js](#)[Descargar](#) [Reiniciar](#) [🔗](#)

```
import { useState } from 'react';

let initialCounters = [
  0, 0, 0
];

export default function CounterList() {
  const [counters, setCounters] = useState(
    initialCounters
  );

  function handleIncrementClick(index) {
```

▼ Mostrar más

Insertar en un *array*

A veces, es posible que desees insertar un elemento en una posición particular que no esté ni al principio ni al final. Para hacer esto, puedes usar la sintaxis de propagación para *arrays* `...` junto con el método `slice()`. El método `slice()` te permite cortar una “rebanada” del *array*. Para insertar un elemento, crearás un *array* que extienda el segmento *antes* del punto de inserción, luego el nuevo elemento y luego el resto del *array* original.

En este ejemplo, el botón “Insertar” siempre inserta en el índice `1`:

App.js

⬇ Descargar ⌛ Reiniciar ↗

```
import { useState } from 'react';

let nextId = 3;
const initialArtists = [
  { id: 0, name: 'Marta Colvin Andrade' },
  { id: 1, name: 'Lamidi Olonade Fakeye'},
  { id: 2, name: 'Louise Nevelson'},
];

export default function List() {
  const [name, setName] = useState('');
  const [artists, setArtists] = useState(
```

▼ Mostrar más

Hacer otros cambios en un *array*

Hay algunas cosas que no puedes hacer con la sintaxis extendida y los métodos que no mutan como `map()` y `filter()`. Por ejemplo, es posible que desees invertir u ordenar un *array*. Los métodos JavaScript `reverse()` y `sort()` mutan el *array* original, por lo que no puedes usarlos directamente.

Sin embargo, puedes copiar el *array* primero y luego realizar cambios en él.

Por ejemplo:

App.js

⬇ Descargar ↺ Reiniciar ↗

```
import { useState } from 'react';

const initialList = [
  { id: 0, title: 'Grandes barrigas' },
  { id: 1, title: 'Paisaje lunar' },
  { id: 2, title: 'Guerreros de terracota' },
];

export default function List() {
  const [list, setList] = useState(initialList);

  function handleClick() {
```

▼ Mostrar más

Aquí, usas la sintaxis de propagación `[...list]` para crear primero una copia del *array* original. Ahora que tienes una copia, puedes usar métodos de mutación como `nextList.reverse()` o `nextList.sort()`, o incluso asignar elementos individuales con `nextList[0] = "algo"`.

Sin embargo, **incluso si copias un *array*, no puedes mutar los elementos existentes *dentro de éste directamente***. Esto se debe a que la copia es superficial: el nuevo *array* contendrá los mismos elementos que el original. Entonces, si modificas un objeto dentro del *array* copiado, estás mutando el estado existente. Por ejemplo, un código como este es un problema.

```
const nextList = [...list];
nextList[0].seen = true; // Problema: muta list[0]
setList(nextList);
```

Aunque `nextList` y `list` son dos *arrays* diferentes, `nextList[0]` y `list[0]` **apuntan al mismo objeto**. Entonces, al cambiar `nextList[0].seen`, está también cambiando `list[0].seen`. ¡Esta es una mutación de estado que debes evitar! Puedes resolver este problema de forma similar a [actualizar objetos JavaScript anidados](#): copiando elementos individuales que deseas cambiar en lugar de mutarlos. Así es cómo.

Actualizar objetos dentro de *arrays*

Los objetos no están *realmente* ubicados “dentro” de los *arrays*. Puede parecer que están “dentro” del código, pero cada objeto en un *array* es un valor separado, al que “apunta” el *array*. Es por eso que debe tener cuidado al cambiar campos

anidados como `list[0]`. ¡La lista de obras de arte de otra persona puede apuntar al mismo elemento del *array*!

Al actualizar el estado anidado, debe crear copias desde el punto en el que desea actualizar y hasta el nivel superior. Veamos cómo funciona esto.

En este ejemplo, dos listas separadas de ilustraciones tienen el mismo estado inicial. Se supone que deben estar aislados, pero debido a una mutación, su estado se comparte accidentalmente y marcar una casilla en una lista afecta a la otra lista:

App.js

⬇ Descargar ↺ Reiniciar ↗

```
import { useState } from 'react';

let nextId = 3;
const initialList = [
  { id: 0, title: 'Grandes barrigas', seen: false },
  { id: 1, title: 'Paisaje lunar', seen: false },
  { id: 2, title: 'Guerreros de terracota', seen: true },
];

export default function BucketList() {
  const [myList, setMyList] = useState(initialList);
  const [yourList, setYourList] = useState(
```

▼ Mostrar más

El problema está en un código como este:

```
const myNextList = [...myList];
const artwork = myNextList.find(a => a.id === artworkId);
artwork.seen = nextSeen; // Problema: muta un elemento existente
setMyList(myNextList);
```

Aunque el *array* `myNextList` en sí mismo es nuevo, los *propios elementos* son los mismos que en el *array* `myList` original. Entonces, cambiar `artwork.seen` cambia el elemento de la obra de arte *original*. Ese elemento de la obra de arte también está en `yourArtworks`, lo que causa el error. Puede ser difícil pensar en errores como este, pero afortunadamente desaparecen si evitas mutar el estado.

Puedes usar `map` para sustituir un elemento antiguo con su versión actualizada sin mutación.

```
setMyList(myList.map(artwork => {
  if (artwork.id === artworkId) {
    // Crea un *nuevo* objeto con cambios
    return { ...artwork, seen: nextSeen };
  } else {
    // No cambia
    return artwork;
  }
}));
```

Aquí, `...` es la sintaxis de propagación de objetos utilizada para [crear una copia de un objeto](#).

Con este enfoque, ninguno de los elementos del estado existentes se modifica y el error se soluciona:

App.js

Descargar Reiniciar

```
import { useState } from 'react';

let nextId = 3;
const initialList = [
  { id: 0, title: 'Grandes barrigas', seen: false },
  { id: 1, title: 'Paisaje lunar', seen: false },
  { id: 2, title: 'Guerreros de terracota', seen: true },
];

export default function BucketList() {
  const [myList, setMyList] = useState(initialList);
  const [yourList, setYourList] = useState(
```

▼ Mostrar más

En general, **solo debes mutar objetos que acabas de crear**. Si estuvieras insertando una *nueva* obra de arte, podrías mutarla, pero si se trata de algo que ya está en el estado, debes hacer una copia.

Escribe una lógica de actualización concisa con Immer

Actualizar *arrays* anidados sin mutación puede volverse un poco repetitivo. [Al igual que con los objetos](#):

- En general, no deberías necesitar actualizar el estado más de un par de niveles de profundidad. Si tus objetos de estado son muy profundos, es posible que desees [reestructurarlos de manera diferente](#) para que sean planos.
- Si no deseas cambiar la estructura de tu estado, puedes preferir usar [Immer](#), que te permite escribir usando la sintaxis conveniente, pero que realiza mutaciones, y se encarga de producir las copias por ti.

Aquí está el ejemplo de una Lista de deseos de arte reescrito con Immer:

[package.json](#) [App.js](#)

↻ Reiniciar [🔗](#)

```
{
  "dependencies": {
    "immer": "1.7.3",
    "react": "latest",
    "react-dom": "latest",
    "react-scripts": "latest",
    "use-immer": "0.5.1"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test --env=jsdom",
```

Ten en cuenta cómo con Immer, **la mutación como** `artwork.seen = nextSeen` **ahora está bien:**

```
updateMyTodos(draft => {  
  const artwork = draft.find(a => a.id === artworkId);  
  artwork.seen = nextSeen;  
});
```

Esto se debe a que no está mutando el estado *original*, sino que está mutando un objeto `draft` especial proporcionado por Immer. Del mismo modo, puedes aplicar métodos de mutación como `push()` y `pop()` al contenido del `draft`.

Tras bambalinas, Immer siempre construye el siguiente estado desde cero de acuerdo con los cambios que ha realizado en el `draft`. Esto mantiene tus controladores de eventos muy concisos sin mutar nunca el estado.

Recapitulación

- Puedes poner *arrays* en el estado, pero no puedes cambiarlos.
- En lugar de mutar un *array*, crea una *nueva* versión y actualiza el estado.
- Puedes usar la sintaxis de propagación `[...arr, newItem]` para crear *arrays* con nuevos elementos.
- Puedes usar `filter()` y `map()` para crear nuevos *arrays* con elementos filtrados o transformados.
- Puedes usar Immer para mantener tu código conciso.

Prueba algunos desafíos

1. Actualizar un artículo en el carrito de compras

2. Eliminar un artículo



Desafío 1 de 4:

Actualizar un artículo en el carrito de compras

Completa la lógica `handleIncreaseClick` para que al presionar "+" aumente el número correspondiente:

App.js

⬇ Descargar ↺ Reiniciar ↗

```
import { useState } from 'react';

const initialProducts = [{
  id: 0,
  name: 'Baklava',
  count: 1,
}, {
  id: 1,
  name: 'Queso',
  count: 5,
}, {
  id: 2,
```

▼ Mostrar más

[📄 Mostrar solución](#)[Próximo desafío](#)

ANTERIOR
< [Actualizar objetos en el estado](#)

SIGUIENTE
[Gestión del estado](#) >

 Meta Open Source

Copyright © Meta Platforms, Inc

uwu?

Aprender React

[Inicio rápido](#)

[Instalación](#)

[Describir la UI](#)

[Agregar interactividad](#)

[Gestión del estado](#)

[Puertas de escape](#)

Referencia de la API

[React APIs](#)

[React DOM APIs](#)

Comunidad

[Código de conducta](#)

[Conoce al equipo](#)

[Contribuidores de la documentación](#)

[Agradecimientos](#)

Más

[Blog](#)

[React Native](#)

[Privacidad](#)

[Términos](#)

